CAPÍTULO 7

Gestión de la memoria

7.1. Requisitos de la gestión de la memoria

7.2. Particionamiento de la memoria

7.3. Paginación

7.4. Segmentación

7.5. Resumen

7.6. Lecturas recomendadas

7.7. Términos clave, cuestiones de revisión y problemas

Apéndice 7A Carga y enlace

En un sistema monoprogramado, la memoria se divide en dos partes: una parte para el sistema operativo (monitor residente, núcleo) y una parte para el programa actualmente en ejecución. En un sistema multiprogramado, la parte de «usuario» de la memoria se debe subdividir posteriormente para acomodar múltiples procesos. El sistema operativo es el encargado de la tarea de subdivisión y a esta tarea se le denomina ***gestión de la memoria****.*

Una gestión de la memoria efectiva es vital en un sistema multiprogramado. Si sólo unos pocos procesos se encuentran en memoria, entonces durante una gran parte del tiempo todos los procesos esperarían por operaciones de E/S y el procesador estaría ocioso. Por tanto, es necesario asignar la memoria para asegurar una cantidad de procesos listos que consuman el tiempo de procesador disponible.

Comenzaremos este capítulo con una descripción de los requisitos que la gestión de la memoria pretende satisfacer. A continuación, se mostrará la tecnología de la gestión de la memoria, analizando una variedad de esquemas simples que se han utilizado. El capítulo se enfoca en el requisito de que un programa se debe cargar en memoria principal para ejecutarse. Esta discusión introduce algunos de los principios fundamentales de la gestión de la memoria.

7.1. REQUISITOS DE LA GESTIÓN DE LA MEMORIA

Mientras se analizan varios mecanismos y políticas asociados con la gestión de la memoria, es útil mantener en mente los requisitos que la gestión de la memoria debe satisfacer. [LIST93] sugiere cinco requisitos:

- Reubicación.
- Protección.
- Compartición.
- Organización lógica.
- Organización física.

REUBICACIÓN

En un sistema multiprogramado, la memoria principal disponible se comparte generalmente entre varios procesos. Normalmente, no es posible que el programador sepa anticipadamente qué programas residirán en memoria principal en tiempo de ejecución de su programa. Adicionalmente, sería bueno poder intercambiar procesos en la memoria principal para maximizar la utilización del procesador, proporcionando un gran número de procesos para la ejecución. Una vez que un programa se ha llevado al disco, sería bastante limitante tener que colocarlo en la misma región de memoria principal donde se hallaba anteriormente, cuando éste se trae de nuevo a la memoria. Por el contrario, podría ser necesario **reubicar** el proceso a un área de memoria diferente.

Por tanto, no se puede conocer de forma anticipada dónde se va a colocar un programa y se debe permitir que los programas se puedan mover en la memoria principal, debido al intercambio o *swap*. Estos hechos ponen de manifiesto algunos aspectos técnicos relacionados con el direccionamiento, como se muestra en la Figura 7.1. La figura representa la imagen de un proceso. Por razones de simplicidad, se asumirá que la imagen de un proceso ocupa una región contigua de la memoria principal. Claramente, el sistema operativo necesitará conocer la ubicación de la información de control

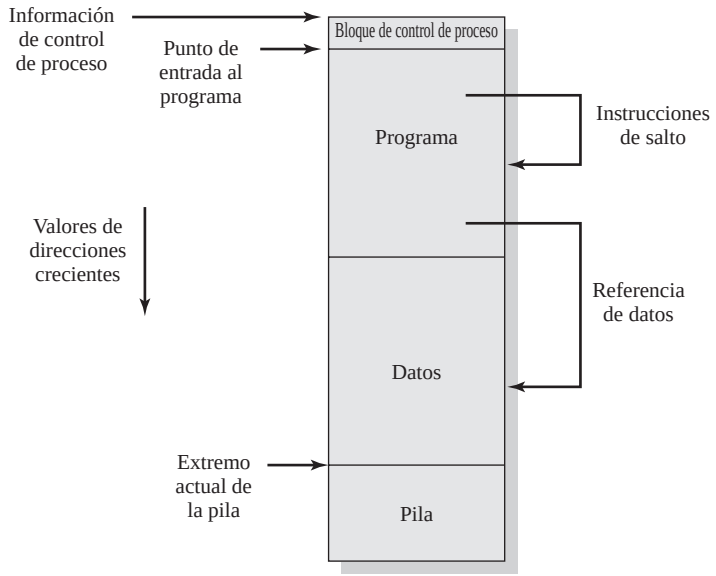


Figura 7.1. Requisitos de direccionamiento para un proceso.

del proceso y de la pila de ejecución, así como el punto de entrada que utilizará el proceso para iniciar la ejecución. Debido a que el sistema operativo se encarga de gestionar la memoria y es responsable de traer el proceso a la memoria principal, estas direcciones son fáciles de adquirir. Adicionalmente, sin embargo, el procesador debe tratar con referencias de memoria dentro del propio programa. Las instrucciones de salto contienen una dirección para referenciar la instrucción que se va a ejecutar a continuación. Las instrucciones de referencia de los datos contienen la dirección del byte o palabra de datos referenciados. De alguna forma, el hardware del procesador y el software del sistema operativo deben poder traducir las referencias de memoria encontradas en el código del programa en direcciones de memoria físicas, que reflejan la ubicación actual del programa en la memoria principal.

PROTECCIÓN

Cada proceso debe protegerse contra interferencias no deseadas por parte de otros procesos, sean accidentales o intencionadas. Por tanto, los programas de otros procesos no deben ser capaces de referenciar sin permiso posiciones de memoria de un proceso, tanto en modo lectura como escritura. Por un lado, lograr los requisitos de la reubicación incrementa la dificultad de satisfacer los requisitos de protección. Más aún, la mayoría de los lenguajes de programación permite el cálculo dinámico de direcciones en tiempo de ejecución (por ejemplo, calculando un índice de posición en un vector o un puntero a una estructura de datos). Por tanto, todas las referencias de memoria generadas por un proceso deben comprobarse en tiempo de ejecución para poder asegurar que se refieren sólo al espacio de memoria asignado a dicho proceso. Afortunadamente, se verá que los mecanismos que dan soporte a la reasignación también dan soporte al requisito de protección.

Normalmente, un proceso de usuario no puede acceder a cualquier porción del sistema operativo, ni al código ni a los datos. De nuevo, un programa de un proceso no puede saltar a una instrucción de otro proceso. Sin un trato especial, un programa de un proceso no puede acceder al área de datos de otro proceso. El procesador debe ser capaz de abortar tales instrucciones en el punto de ejecución.

Obsérvese que los requisitos de protección de memoria deben ser satisfechos por el procesador (hardware) en lugar del sistema operativo (software). Esto es debido a que el sistema operativo no puede anticipar todas las referencias de memoria que un programa hará. Incluso si tal anticipación fuera posible, llevaría demasiado tiempo calcularlo para cada programa a fin de comprobar la violación de referencias de la memoria. Por tanto, sólo es posible evaluar la permisibilidad de una referencia (acceso a datos o salto) en tiempo de ejecución de la instrucción que realiza dicha referencia. Para llevar a cabo esto, el hardware del procesador debe tener esta capacidad.

COMPARTICIÓN

Cualquier mecanismo de protección debe tener la flexibilidad de permitir a varios procesos acceder a la misma porción de memoria principal. Por ejemplo, si varios programas están ejecutando el mismo programa, es ventajoso permitir que cada proceso pueda acceder a la misma copia del programa en lugar de tener su propia copia separada. Procesos que estén cooperando en la misma tarea podrían necesitar compartir el acceso a la misma estructura de datos. Por tanto, el sistema de gestión de la memoria debe permitir el acceso controlado a áreas de memoria compartidas sin comprometer la protección esencial. De nuevo, se verá que los mecanismos utilizados para dar soporte a la reubicación soportan también capacidades para la compartición.

ORGANIZACIÓN LÓGICA

Casi invariablemente, la memoria principal de un computador se organiza como un espacio de almacenamiento lineal o unidimensional, compuesto por una secuencia de bytes o palabras. A nivel físico, la memoria secundaria está organizada de forma similar. Mientras que esta organización es similar al hardware real de la máquina, no se corresponde a la forma en la cual los programas se construyen normalmente. La mayoría de los programas se organizan en módulos, algunos de los cuales no se pueden modificar (sólo lectura, sólo ejecución) y algunos de los cuales contienen datos que se pueden modificar. Si el sistema operativo y el hardware del computador pueden tratar de forma efectiva los programas de usuarios y los datos en la forma de módulos de algún tipo, entonces se pueden lograr varias ventajas:

1. Los módulos se pueden escribir y compilar independientemente, con todas las referencias de un módulo desde otro resueltas por el sistema en tiempo de ejecución.
2. Con una sobrecarga adicional modesta, se puede proporcionar diferentes grados de protección a los módulos (sólo lectura, sólo ejecución).
3. Es posible introducir mecanismos por los cuales los módulos se pueden compartir entre los procesos. La ventaja de proporcionar compartición a nivel de módulo es que se corresponde con la forma en la que el usuario ve el problema, y por tanto es fácil para éste especificar la compartición deseada.

La herramienta que más adecuadamente satisface estos requisitos es la segmentación, que es una de las técnicas de gestión de la memoria exploradas en este capítulo.

ORGANIZACIÓN FÍSICA

Como se discute en la Sección 1.5, la memoria del computador se organiza en al menos dos niveles, conocidos como memoria principal y memoria secundaria. La memoria principal proporciona acceso

rápido a un coste relativamente alto. Adicionalmente, la memoria principal es volátil; es decir, no proporciona almacenamiento permanente. La memoria secundaria es más lenta y más barata que la memoria principal y normalmente no es volátil. Por tanto, la memoria secundaria de larga capacidad puede proporcionar almacenamiento para programas y datos a largo plazo, mientras que una memoria principal más pequeña contiene programas y datos actualmente en uso.

En este esquema de dos niveles, la organización del flujo de información entre la memoria principal y secundaria supone una de las preocupaciones principales del sistema. La responsabilidad para este flujo podría asignarse a cada programador en particular, pero no es practicable o deseable por dos motivos:

1. La memoria principal disponible para un programa más sus datos podría ser insuficiente. En este caso, el programador debería utilizar una técnica conocida como **superposición** (***overlaying***), en la cual los programas y los datos se organizan de tal forma que se puede asignar la misma región de memoria a varios módulos, con un programa principal responsable para intercambiar los módulos entre disco y memoria según las necesidades. Incluso con la ayuda de herramientas de compilación, la programación con *overlays* malgasta tiempo del programador.
2. En un entorno multiprogramado, el programador no conoce en tiempo de codificación cuánto espacio estará disponible o dónde se localizará dicho espacio.

Por tanto, está claro que la tarea de mover la información entre los dos niveles de la memoria debería ser una responsabilidad del sistema. Esta tarea es la esencia de la gestión de la memoria.

7.2. PARTICIONAMIENTO DE LA MEMORIA

La operación principal de la gestión de la memoria es traer los procesos a la memoria principal para que el procesador los pueda ejecutar. En casi todos los sistemas multiprogramados modernos, esto implica el uso de un esquema sofisticado denominado memoria virtual. Por su parte, la memoria virtual se basa en una o ambas de las siguientes técnicas básicas: segmentación y paginación. Antes de fijarse en estas técnicas de memoria virtual, se debe preparar el camino, analizando técnicas más sencillas que no utilizan memoria virtual (Tabla 7.1). Una de estas técnicas, el particionamiento, se ha utilizado en algunas variantes de ciertos sistemas operativos ahora obsoletos. Las otras dos técnicas, paginación sencilla y segmentación sencilla, no son utilizadas de forma aislada. Sin embargo, quedará más clara la discusión de la memoria virtual si se analizan primero estas dos técnicas sin tener en cuenta consideraciones de memoria virtual.

PARTICIONAMIENTO FIJO

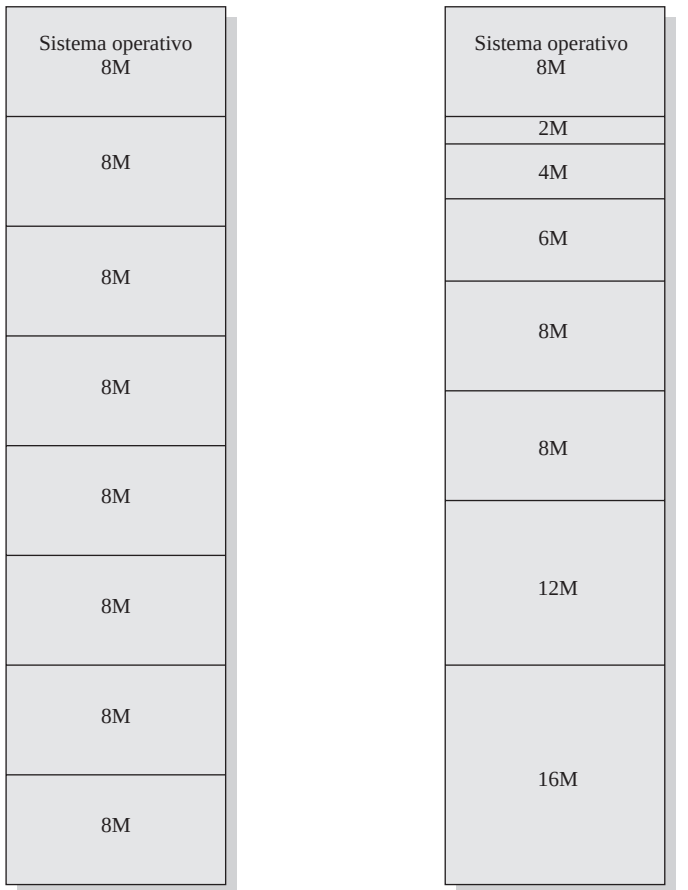
En la mayoría de los esquemas para gestión de la memoria, se puede asumir que el sistema operativo ocupa alguna porción fija de la memoria principal y que el resto de la memoria principal está disponible para múltiples procesos. El esquema más simple para gestionar la memoria disponible es repartirla en regiones con límites fijos.

Tamaños de partición

La Figura 7.2 muestra ejemplos de dos alternativas para el particionamiento fijo. Una posibilidad consiste en hacer uso de particiones del mismo tamaño. En este caso, cualquier proceso cuyo tamaño

Tabla 7.1. Técnicas de gestión de memoria.

Técnica	Descripción	Virtudes	Defectos
Particionamiento fijo	La memoria principal se divide en particiones estáticas en tiempo de generación del sistema. Un proceso se puede cargar en una partición con igual o superior tamaño.	Sencilla de implementar; poca sobrecarga para el sistema operativo.	Uso ineficiente de la memoria, debido a la fragmentación interna; debe fijarse el número máximo de procesos activos.
Particionamiento dinámico	Las particiones se crean de forma dinámica, de tal forma que cada proceso se carga en una partición del mismo tamaño que el proceso.	No existe fragmentación interna; uso más eficiente de memoria principal.	Uso ineficiente del procesador, debido a la necesidad de compactación para evitar la fragmentación externa.
Paginación sencilla	La memoria principal se divide en marcos del mismo tamaño. Cada proceso se divide en páginas del mismo tamaño que los marcos. Un proceso se carga a través de la carga de todas sus páginas en marcos disponibles, no necesariamente contiguos.	No existe fragmentación externa.	Una pequeña cantidad de fragmentación interna.
Segmentación sencilla	Cada proceso se divide en segmentos. Un proceso se carga cargando todos sus segmentos en particiones dinámicas, no necesariamente contiguas.	No existe fragmentación interna; mejora la utilización de la memoria y reduce la sobrecargada respecto al particionamiento dinámico.	Fragmentación externa.
Paginación con memoria virtual	Exactamente igual que la paginación sencilla, excepto que no es necesario cargar todas las páginas de un proceso. Las páginas no residentes se traen bajo demanda de forma automática.	No existe fragmentación externa; mayor grado de multiprogramación; gran espacio de direcciones virtuales.	Sobrecarga por la gestión compleja de la memoria.
Segmentación con memoria virtual	Exactamente igual que la segmentación, excepto que no es necesario cargar todos los segmentos de un proceso. Los segmentos no residentes se traen bajo demanda de forma automática.	No existe fragmentación interna; mayor grado de multiprogramación; gran espacio de direcciones virtuales; soporte a protección y compartición.	Sobrecarga por la gestión compleja de la memoria.



(a) Participaciones de igual tamaño (b) Participaciones de distinto tamaño

Figura 7.2. Ejemplo de particionamiento fijo de una memoria de 64 Mbytes.

es menor o igual que el tamaño de partición puede cargarse en cualquier partición disponible. Si todas las particiones están llenas y no hay ningún proceso en estado Listo o Ejecutando, el sistema operativo puede mandar a *swap* a un proceso de cualquiera de las particiones y cargar otro proceso, de forma que el procesador tenga trabajo que realizar.

Existen dos dificultades con el uso de las particiones fijas del mismo tamaño:

- Un programa podría ser demasiado grande para caber en una partición. En este caso, el programador debe diseñar el programa con el uso de *overlays*, de forma que sólo se necesite una porción del programa en memoria principal en un momento determinado. Cuando se necesita un módulo que no está presente, el programa de usuario debe cargar dicho módulo en la partición del programa, superponiéndolo (*overlaying*) a cualquier programa o datos que haya allí.
- La utilización de la memoria principal es extremadamente ineficiente. Cualquier programa, sin importar lo pequeño que sea, ocupa una partición entera. En el ejemplo, podría haber un programa cuya longitud es menor que 2 Mbytes; ocuparía una partición de 8 Mbytes cuando se lleva a la memoria. Este fenómeno, en el cual hay espacio interno malgastado debido al hecho

de que el bloque de datos cargado es menor que la partición, se conoce con el nombre de **fragmentación interna**.

Ambos problemas se pueden mejorar, aunque no resolver, utilizando particiones de tamaño diferente (Figura 7.2b). En este ejemplo, los programas de 16 Mbytes se pueden acomodar sin *overlays*. Las particiones más pequeñas de 8 Mbytes permiten que los programas más pequeños se puedan acomodar sin menor fragmentación interna.

Algoritmo de ubicación

Con particiones del mismo tamaño, la ubicación de los procesos en memoria es trivial. En cuanto haya una partición disponible, un proceso se carga en dicha partición. Debido a que todas las particiones son del mismo tamaño, no importa qué partición se utiliza. Si todas las particiones se encuentran ocupadas por procesos que no están listos para ejecutar, entonces uno de dichos procesos debe llevarse a disco para dejar espacio para un nuevo proceso. Cuál de los procesos se lleva a disco es una decisión de planificación; este tema se describe en la Parte Cuatro.

Con particiones de diferente tamaño, hay dos formas posibles de asignar los procesos a las particiones. La forma más sencilla consiste en asignar cada proceso a la partición más pequeña dentro de la cual cabe[1]. En este caso, se necesita una cola de planificación para cada partición, que mantenga procesos en disco destinados a dicha partición (Figura 7.3a). La ventaja de esta técnica es que los procesos siempre se asignan de tal forma que se minimiza la memoria malgastada dentro de una partición (fragmentación interna).

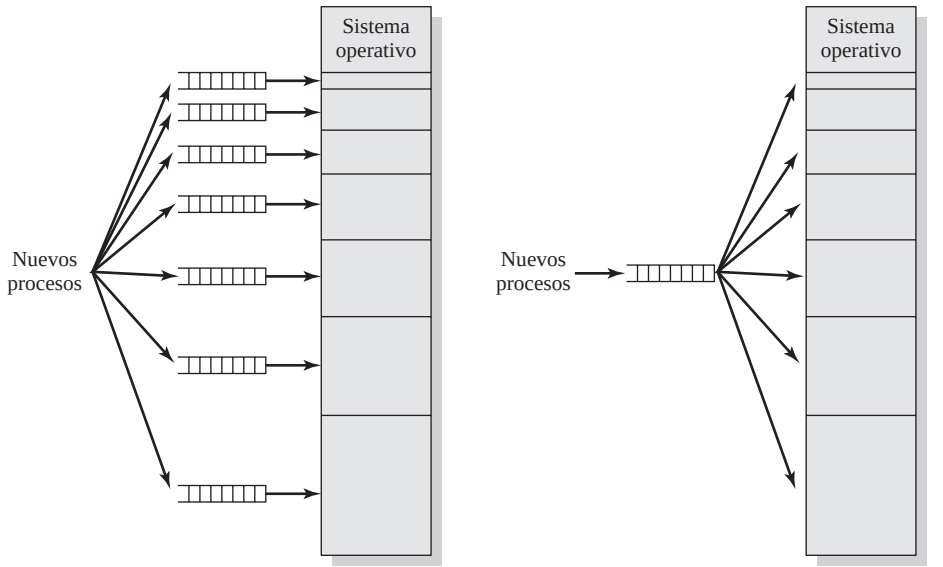
Aunque esta técnica parece óptima desde el punto de vista de una partición individual, no es óptima desde el punto de vista del sistema completo. En la Figura 7.2b, por ejemplo, se considera un caso en el que no haya procesos con un tamaño entre 12 y 16M en un determinado instante de tiempo. En este caso, la partición de 16M quedará sin utilizarse, incluso aunque se puede asignar dicha partición a algunos procesos más pequeños. Por tanto, una técnica óptima sería emplear una única cola para todos los procesos (Figura 7.3b). En el momento de cargar un proceso en la memoria principal, se selecciona la partición más pequeña disponible que puede albergar dicho proceso. Si todas las particiones están ocupadas, se debe llevar a cabo una decisión para enviar a *swap* a algún proceso. Tiene preferencia a la hora de ser expulsado a disco el proceso que ocupe la partición más pequeña que pueda albergar al proceso entrante. Es también posible considerar otros factores, como la prioridad o una preferencia por expulsar a disco procesos bloqueados frente a procesos listos.

El uso de particiones de distinto tamaño proporciona un grado de flexibilidad frente a las particiones fijas. Adicionalmente, se puede decir que los esquemas de particiones fijas son relativamente sencillos y requieren un soporte mínimo por parte del sistema operativo y una sobrecarga de procesamiento mínimo. Sin embargo, tiene una serie de desventajas:

- El número de particiones especificadas en tiempo de generación del sistema limita el número de proceso activos (no suspendidos) del sistema.
- Debido a que los tamaños de las particiones son preestablecidos en tiempo de generación del sistema, los trabajos pequeños no utilizan el espacio de las particiones eficientemente. En un entorno donde el requisito de almacenamiento principal de todos los trabajos se conoce de an-

---

[1] Se asume que se conoce el tamaño máximo de memoria que un proceso requerirá. No siempre es el caso. Si no se sabe lo que un proceso puede ocupar, la única alternativa es un esquema de *overlays* o el uso de memoria virtual.



(a) Una cola de procesos por participación (b) Un única cola

Figura 7.3. Asignación de memoria para particionamiento fijo.

temano, esta técnica puede ser razonable, pero en la mayoría de los casos, se trata de una técnica ineficiente.

El uso de particionamiento fijo es casi desconocido hoy en día. Un ejemplo de un sistema operativo exitoso que sí utilizó esta técnica fue un sistema operativo de los primeros *mainframes* de IBM, el sistema operativo OS/MFT (*Multiprogramming with a Fixed Number of Tasks*; Multiprogramado con un número fijo de tareas).

PARTICIONAMIENTO DINÁMICO

Para vencer algunas de las dificultades con particionamiento fijo, se desarrolló una técnica conocida como particionamiento dinámico. De nuevo, esta técnica se ha sustituido por técnicas de gestión de memoria más sofisticadas. Un sistema operativo importante que utilizó esta técnica fue el sistema operativo de *mainframes* de IBM, el sistema operativo OS/MVT (*Multiprogramming with a Variable Number of Tasks*; Multiprogramado con un número variable de tareas).

Con particionamiento dinámico, las particiones son de longitud y número variable. Cuando se lleva un proceso a la memoria principal, se le asigna exactamente tanta memoria como requiera y no más. Un ejemplo, que utiliza 64 Mbytes de memoria principal, se muestra en la Figura 7.4. Inicialmente, la memoria principal está vacía, excepto por el sistema operativo (a). Los primeros tres procesos se cargan justo donde el sistema operativo finaliza y ocupando el espacio justo para cada proceso (b, c, d). Esto deja un «hueco» al final de la memoria que es demasiado pequeño para un cuarto proceso. En algún momento, ninguno de los procesos que se encuentran en memoria está disponible. El sistema operativo lleva el proceso 2 al disco (e), que deja suficiente espacio para cargar un nuevo proceso, el proceso 4 (f). Debido a que el proceso 4 es más pequeño que el proceso 2, se crea otro pequeño hueco. Posteriormente, se alcanza un punto en el cual ninguno de los procesos de la memoria principal está listo, pero el proceso 2, en estado Listo-Suspendido, está disponible. Porque no hay es-

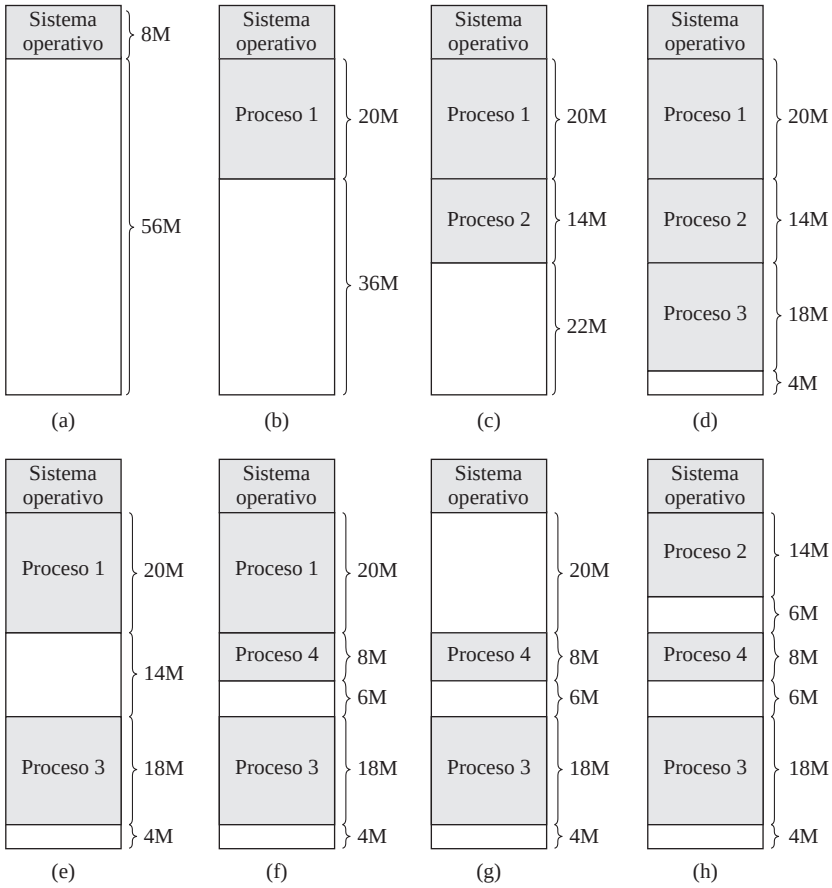


Figura 7.4. El efecto del particionamiento dinámico.

pacio suficiente en la memoria para el proceso 2, el sistema operativo lleva a disco el proceso 1 (g) y lleva a la memoria el proceso 2 (h).

Como muestra este ejemplo, el método comienza correctamente, pero finalmente lleva a una situación en la cual existen muchos huecos pequeños en la memoria. A medida que pasa el tiempo, la memoria se fragmenta cada vez más y la utilización de la memoria se decrementa. Este fenómeno se conoce como **fragmentación externa**, indicando que la memoria que es externa a todas las particiones se fragmenta de forma incremental, por contraposición a lo que ocurre con la fragmentación interna, descrita anteriormente.

Una técnica para eliminar la fragmentación externa es la **compactación**: de vez en cuando, el sistema operativo desplaza los procesos en memoria, de forma que se encuentren contiguos y de este modo toda la memoria libre se encontrará unida en un bloque. Por ejemplo, en la Figura 7.4R, la compactación permite obtener un bloque de memoria libre de longitud 16M. Esto sería suficiente para cargar un proceso adicional. La desventaja de la compactación es el hecho de que se trata de un procedimiento que consume tiempo y malgasta tiempo de procesador. Obsérvese que la compactación requiere la capacidad de reubicación dinámica. Es decir, debe ser posible mover un programa desde una región a otra en la memoria principal sin invalidar las referencias de la memoria de cada programa (*véase* Apéndice 7A).

Algoritmo de ubicación

Debido a que la compactación de memoria consume una gran cantidad de tiempo, el diseñador del sistema operativo debe ser inteligente a la hora de decidir cómo asignar la memoria a los procesos (cómo eliminar los huecos). A la hora de cargar o intercambiar un proceso a la memoria principal, y siempre que haya más de un bloque de memoria libre de suficiente tamaño, el sistema operativo debe decidir qué bloque libre asignar.

Tres algoritmos de colocación que pueden considerarse son mejor-ajuste (*best-fit*), primer-ajuste(*first-fit*) y siguiente-ajuste (*next-fit*). Todos, por supuesto, están limitados a escoger entre los bloques libres de la memoria principal que son iguales o más grandes que el proceso que va a llevarse a la memoria. **Mejor-ajuste** escoge el bloque más cercano en tamaño a la petición. **Primer-ajuste** comienza a analizar la memoria desde el principio y escoge el primer bloque disponible que sea suficientemente grande. **Siguiente-ajuste** comienza a analizar la memoria desde la última colocación y elige el siguiente bloque disponible que sea suficientemente grande.

La Figura 7.5a muestra un ejemplo de configuración de memoria después de un número de colocaciones e intercambios a disco. El último bloque que se utilizó fue un bloque de 22 Mbytes del cual se crea una partición de 14 Mbytes. La Figura 7.5b muestra la diferencia entre los algoritmos de mejor-, primer- y siguiente- ajuste a la hora de satisfacer una petición de asignación de 16 Mbytes. Me-

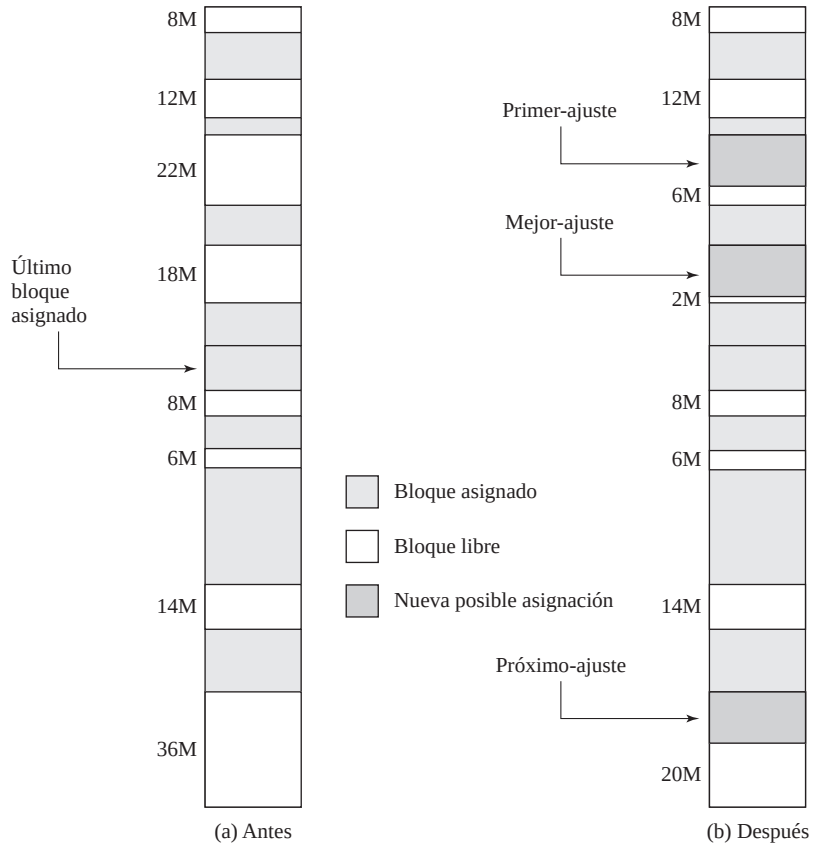


Figura 7.5. Ejemplo de configuración de la memoria antes y después de la asignación de un bloque de 16 Mbytes.

jor-ajuste busca la lista completa de bloques disponibles y hace uso del bloque de 18 Mbytes, dejando un fragmento de 2 Mbytes. *Primer-ajuste* provoca un fragmento de 6 Mbytes, y *siguiente-ajuste* provoca un fragmento de 20 Mbytes.

Cuál de estas técnicas es mejor depende de la secuencia exacta de intercambio de procesos y del tamaño de dichos procesos. Sin embargo, se pueden hacer algunos comentarios (*véase* también [BREN89], [SHOR75] y [BAYS77]). El algoritmo primer-ajuste no es sólo el más sencillo, sino que normalmente es también el mejor y más rápido. El algoritmo siguiente-ajuste tiende a producir resultados ligeramente peores que el primer-ajuste. El algoritmo siguiente-ajuste llevará más frecuentemente a una asignación de un bloque libre al final de la memoria. El resultado es que el bloque más grande de memoria libre, que normalmente aparece al final del espacio de la memoria, se divide rápidamente en pequeños fragmentos. Por tanto, en el caso del algoritmo siguiente-ajuste se puede requerir más frecuentemente la compactación. Por otro lado, el algoritmo primer-ajuste puede dejar el final del espacio de almacenamiento con pequeñas particiones libres que necesitan buscarse en cada paso del primer-ajuste siguiente. El algoritmo mejor-ajuste, a pesar de su nombre, su comportamiento normalmente es el peor. Debido a que el algoritmo busca el bloque más pequeño que satisfaga la petición, garantiza que el fragmento que quede sea lo más pequeño posible. Aunque cada petición de memoria siempre malgasta la cantidad más pequeña de la memoria, el resultado es que la memoria principal se queda rápidamente con bloques demasiado pequeños para satisfacer las peticiones de asignación de la memoria. Por tanto, la compactación de la memoria se debe realizar más frecuentemente que con el resto de los algoritmos.

Algoritmo de reemplazamiento

En un sistema multiprogramado que utiliza particionamiento dinámico, puede haber un momento en el que todos los procesos de la memoria principal estén en estado bloqueado y no haya suficiente memoria para un proceso adicional, incluso después de llevar a cabo una compactación. Para evitar malgastar tiempo de procesador esperando a que un proceso se desbloquee, el sistema operativo intercambiará alguno de los procesos entre la memoria principal y disco para hacer sitio a un nuevo proceso o para un proceso que se encuentre en estado Listo-Suspendido. Por tanto, el sistema operativo debe escoger qué proceso reemplazar. Debido a que el tema de los algoritmos de reemplazo se contemplará en detalle respecto a varios esquemas de la memoria virtual, se pospone esta discusión hasta entonces.

SISTEMA BUDDY

Ambos esquemas de particionamiento, fijo y dinámico, tienen desventajas. Un esquema de particionamiento fijo limita el número de procesos activos y puede utilizar el espacio ineficientemente si existe un mal ajuste entre los tamaños de partición disponibles y los tamaños de los procesos. Un esquema de particionamiento dinámico es más complejo de mantener e incluye la sobrecarga de la compactación. Un compromiso interesante es el sistema *buddy* ([KNUT97], [PETE77]).

En un sistema *buddy*, los bloques de memoria disponibles son de tamaño 2^k, $L \leq K \leq U$, donde

2^L = bloque de tamaño más pequeño asignado

2^U = bloque de tamaño mayor asignado; normalmente 2^U es el tamaño de la memoria completa disponible

Para comenzar, el espacio completo disponible se trata como un único bloque de tamaño 2^U. Si se realiza una petición de tamaño *s*, tal que $2^{U-1} < s \leq 2^U$, se asigna el bloque entero. En otro caso, el blo-

que se divide en dos bloques *buddy* iguales de tamaño 2^{U-1}. Si $2^{U-2} < s \leq 2^{U-1}$, entonces se asigna la petición a uno de los otros dos bloques. En otro caso, uno de ellos se divide por la mitad de nuevo. Este proceso continúa hasta que el bloque más pequeño mayor o igual que *s* se genera y se asigna a la petición. En cualquier momento, el sistema *buddy* mantiene una lista de huecos (bloques sin asignar) de cada tamaño 2^i. Un hueco puede eliminarse de la lista (i+1) dividiéndolo por la mitad para crear dos bloques de tamaño 2^i en la lista *i*. Siempre que un par de bloques de la lista *i* no se encuentren asignados, son eliminados de dicha lista y unidos en un único bloque de la lista (i+1). Si se lleva a cabo una petición de asignación de tamaño *k* tal que $2^{i-1} < k \leq 2^i$, se utiliza el siguiente algoritmo recursivo (de [LIST93]) para encontrar un hueco de tamaño 2^i:

```
void obtener_hueco (int i)
{
    if (i==(U+1))
        <fallo>;
    if (<lista_i vacía>)
    {
        obtener_hueco(i+1);
        <dividir hueco en dos buddies>;
        <colocar buddies en lista_i>;
    }
    <tomar primer hueco de la lista_i>;
}
```

La Figura 7.6 muestra un ejemplo que utiliza un bloque inicial de 1 Mbyte. La primera petición, A, es de 100 Kbytes, de tal forma que se necesita un bloque de 128K. El bloque inicial se divide en dos de 512K. El primero de éstos se divide en dos bloques de 256K, y el primero de éstos se divide en dos de 128K, uno de los cuales se asigna a A. La siguiente petición, B, solicita un bloque de 256K. Dicho bloque está disponible y es asignado. El proceso continúa, provocando divisiones y fusiones de bloques cuando se requiere. Obsérvese que cuando se libera E, se unen dos bloques de 128K en un bloque de 256K, que es inmediatamente unido con su bloque compañero (*buddy*).

La Figura 7.7 muestra una representación en forma de árbol binario de la asignación de bloques inmediatamente después de la petición «Liberar B». Los nodos hoja representan el particionamiento de la memoria actual. Si dos bloques son nodos hoja, entonces al menos uno de ellos debe estar asignado; en otro caso, se unen en un bloque mayor.

El sistema *buddy* es un compromiso razonable para eliminar las desventajas de ambos esquemas de particionamiento, fijo y variable, pero en los sistemas operativos contemporáneos, la memoria virtual basada en paginación y segmentación es superior. Sin embargo, el sistema *buddy* se ha utilizado en sistemas paralelos como una forma eficiente de asignar y liberar programas paralelos (por ejemplo, *véase* [JOHN92]). Una forma modificada del sistema *buddy* se utiliza en la asignación de memoria del núcleo UNIX (descrito en el Capítulo 8).

REUBICACIÓN

Antes de considerar formas de tratar los problemas del particionamiento, se va a aclarar un aspecto relacionado con la colocación de los procesos en la memoria. Cuando se utiliza el esquema de particionamiento fijo de la Figura 7.3a, se espera que un proceso siempre se asigne a la misma partición.

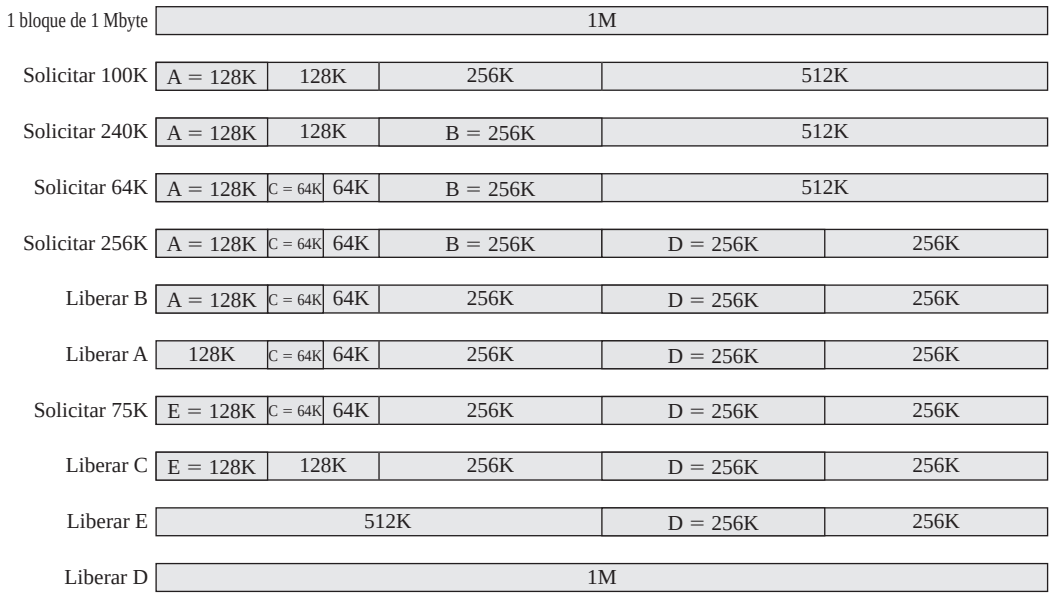


Figura 7.6. Ejemplo de sistema *Buddy.*

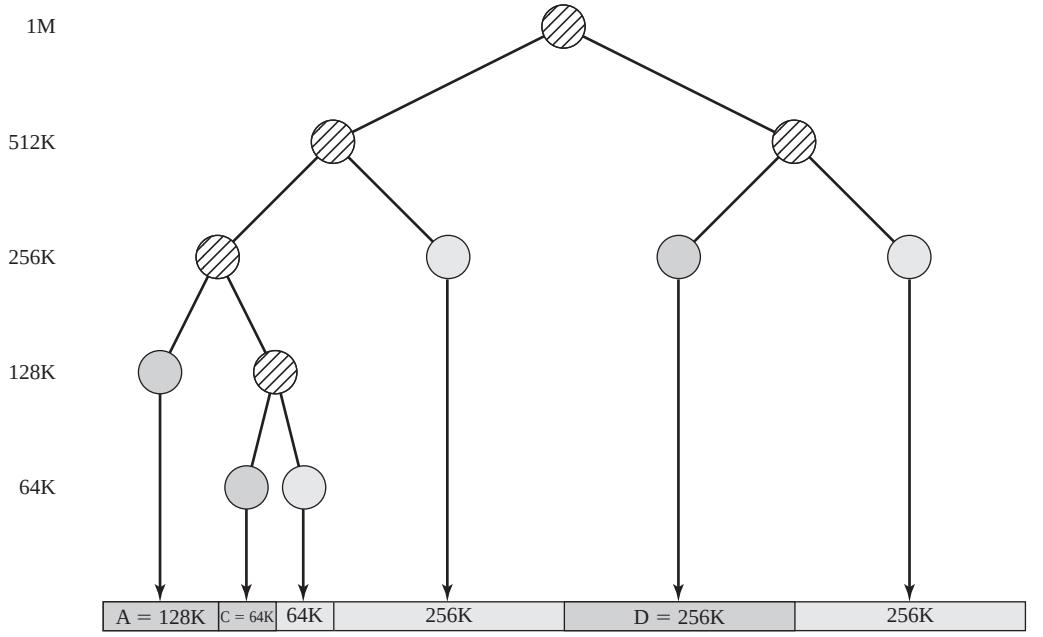


Figura 7.7. Representación en forma de árbol del sistema *Buddy.*

Es decir, sea cual sea la partición seleccionada cuando se carga un nuevo proceso, ésta será la utilizada para el intercambio del proceso entre la memoria y el área de *swap* en disco. En este caso, se utiliza un cargador sencillo, tal y como se describe en el Apéndice 7A: cuando el proceso se carga por

primera vez, todas las referencias de la memoria relativas del código se reemplazan por direcciones de la memoria principal absolutas, determinadas por la dirección base del proceso cargado.

En el caso de particiones de igual tamaño (Figura 7.2), y en el caso de una única cola de procesos para particiones de distinto tamaño (Figura 7.3b), un proceso puede ocupar diferentes particiones durante el transcurso de su ciclo de vida. Cuando la imagen de un proceso se crea por primera vez, se carga en la misma partición de memoria principal. Más adelante, el proceso podría llevarse a disco; cuando se trae a la memoria principal posteriormente, podría asignarse a una partición distinta de la última vez. Lo mismo ocurre en el caso del particionamiento dinámico. Obsérvese en las Figuras 7.4c y h que el proceso 2 ocupa dos regiones diferentes de memoria en las dos ocasiones que se trae a la memoria. Más aún, cuando se utiliza la compactación, los procesos se desplazan mientras están en la memoria principal. Por tanto, las ubicaciones (de las instrucciones y los datos) referenciadas por un proceso no son fijas. Cambiarán cada vez que un proceso se intercambia o se desplaza. Para resolver este problema, se realiza una distinción entre varios tipos de direcciones. Una **dirección lógica** es una referencia a una ubicación de memoria independiente de la asignación actual de datos a la memoria; se debe llevar a cabo una traducción a una dirección física antes de que se alcance el acceso a la memoria. Una **dirección relativa** es un ejemplo particular de dirección lógica, en el que la dirección se expresa como una ubicación relativa a algún punto conocido, normalmente un valor en un registro del procesador. Una **dirección física**, o dirección absoluta, es una ubicación real de la memoria principal.

Los programas que emplean direcciones relativas de memoria se cargan utilizando carga dinámica en tiempo de ejecución (*véase* Apéndice 7A, donde se recoge una discusión). Normalmente, todas las referencias de memoria de los procesos cargados son relativas al origen del programa. Por tanto, se necesita un mecanismo hardware para traducir las direcciones relativas a direcciones físicas de la memoria principal, en tiempo de ejecución de la instrucción que contiene dicha referencia.

La Figura 7.8 muestra la forma en la que se realiza normalmente esta traducción de direcciones. Cuando un proceso se asigna al estado Ejecutando, un registro especial del procesador, algunas veces llamado registro base, carga la dirección inicial del programa en la memoria principal. Existe también un registro «valla», que indica el final de la ubicación del programa; estos valores se establecen cuando el programa se carga en la memoria o cuando la imagen del proceso se lleva a la memoria. A lo largo de la ejecución del proceso, se encuentran direcciones relativas. Éstas incluyen los contenidos del registro de las instrucciones, las direcciones de instrucciones que ocurren en los saltos e instrucciones *call*, y direcciones de datos existentes en instrucciones de carga y almacenamiento. El procesador manipula cada dirección relativa, a través de dos pasos. Primero, el valor del registro base se suma a la dirección relativa para producir una dirección absoluta. Segundo, la dirección resultante se compara con el valor del registro «valla». Si la dirección se encuentra dentro de los límites, entonces se puede llevar a cabo la ejecución de la instrucción. En otro caso, se genera una interrupción, que debe manejar el sistema operativo de algún modo.

El esquema de la Figura 7.8 permite que se traigan a memoria los programas y que se lleven a disco, a lo largo de la ejecución. También proporciona una medida de protección: cada imagen del proceso está aislada mediante los contenidos de los registros base y valla. Además, evita accesos no autorizados por parte de otros procesos.

7.3. PAGINACIÓN

Tanto las particiones de tamaño fijo como variable son ineficientes en el uso de la memoria; los primeros provocan fragmentación interna, los últimos fragmentación externa. Supóngase, sin embargo, que la memoria principal se divide en porciones de tamaño fijo relativamente pequeños, y que cada proceso también se divide en porciones pequeñas del mismo tamaño fijo. A dichas porciones del pro-

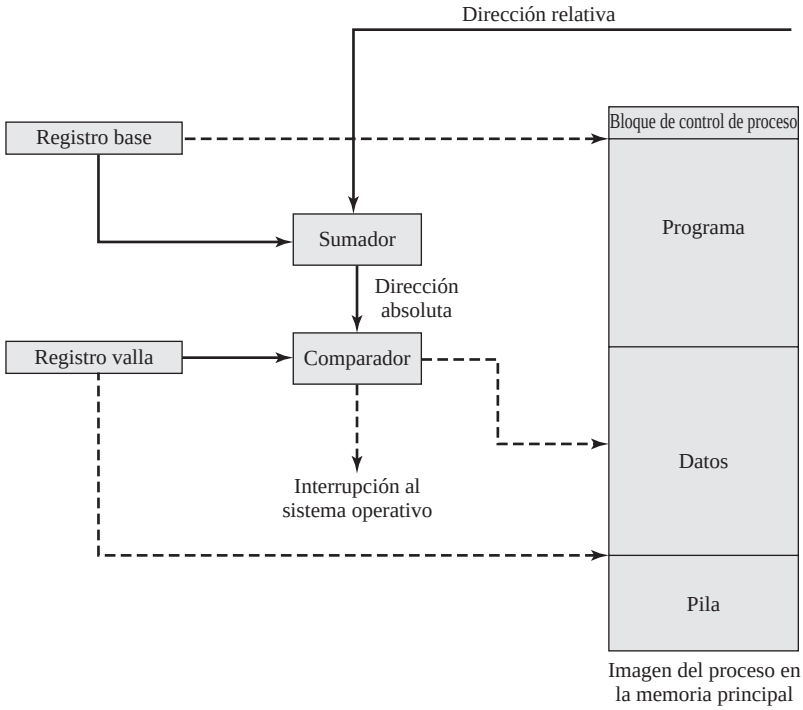


Figura 7.8. Soporte hardware para la reubicación.

ceso, conocidas como **páginas**, se les asigna porciones disponibles de memoria, conocidas como **marcos**, o marcos de páginas. Esta sección muestra que el espacio de memoria malgastado por cada proceso debido a la fragmentación interna corresponde sólo a una fracción de la última página de un proceso. No existe fragmentación externa.

La Figura 7.9 ilustra el uso de páginas y marcos. En un momento dado, algunos de los marcos de la memoria están en uso y algunos están libres. El sistema operativo mantiene una lista de marcos libres. El proceso A, almacenado en disco, está formado por cuatro páginas. En el momento de cargar este proceso, el sistema operativo encuentra cuatro marcos libres y carga las cuatro páginas del proceso A en dichos marcos (Figura 7.9b). El proceso B, formado por tres páginas, y el proceso C, formado por cuatro páginas, se cargan a continuación. En ese momento, el proceso B se suspende y deja la memoria principal. Posteriormente, todos los procesos de la memoria principal se bloquean, y el sistema operativo necesita traer un nuevo proceso, el proceso D, que está formado por cinco páginas.

Ahora supóngase, como en este ejemplo, que no hay suficientes marcos contiguos sin utilizar para ubicar un proceso. ¿Esto evitaría que el sistema operativo cargara el proceso D? La respuesta es no, porque una vez más se puede utilizar el concepto de dirección lógica. Un registro base sencillo de direcciones no basta en esta ocasión. En su lugar, el sistema operativo mantiene una **tabla de páginas** por cada proceso. La tabla de páginas muestra la ubicación del marco por cada página del proceso. Dentro del programa, cada dirección lógica está formada por un número de página y un desplazamiento dentro de la página. Es importante recordar que en el caso de una partición simple, una dirección lógica es la ubicación de una palabra relativa al comienzo del programa; el procesador la traduce en una dirección física. Con paginación, la traducción de direcciones lógicas a físicas las realiza también el hardware del procesador. Ahora el procesador debe conocer cómo acceder a la ta-

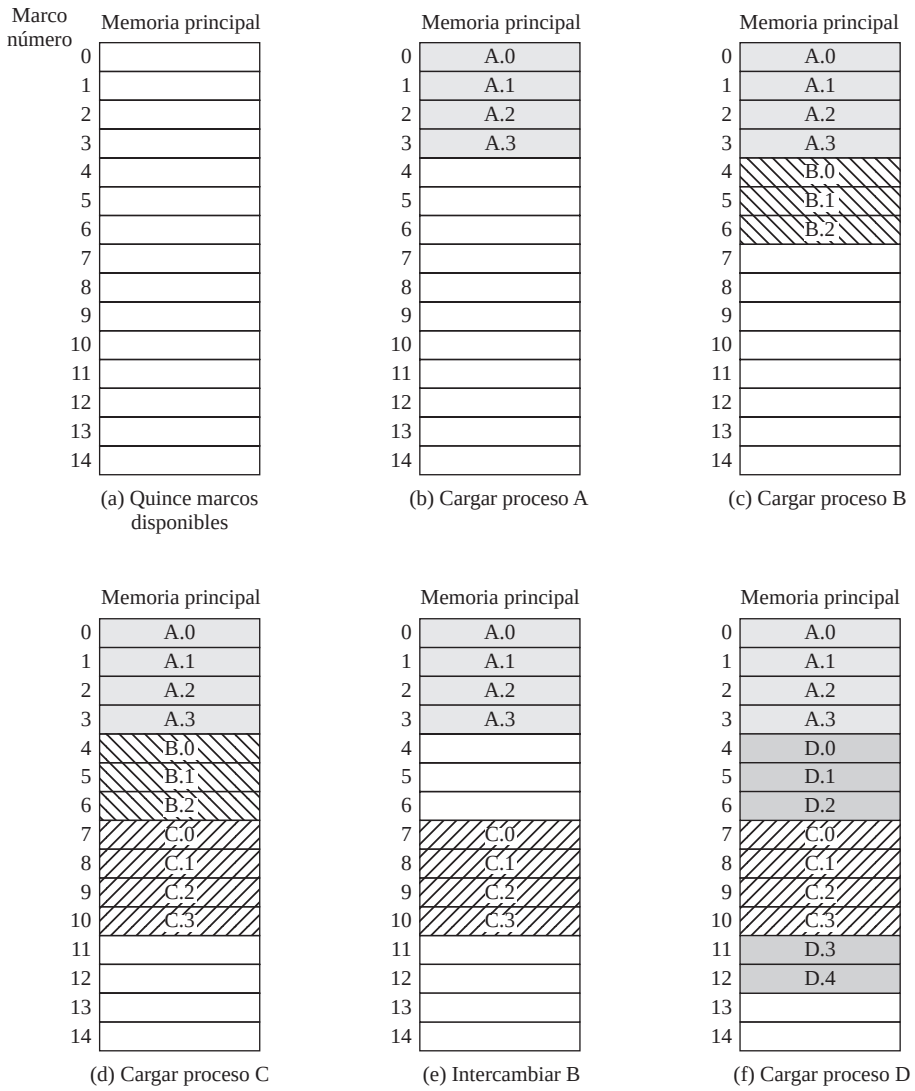


Figura 7.9. Asignación de páginas de proceso a marcos libres.

bla de páginas del proceso actual. Presentado como una dirección lógica (número de página, desplazamiento), el procesador utiliza la tabla de páginas para producir una dirección física (número de marco, desplazamiento).

Continuando con nuestro ejemplo, las cinco páginas del proceso D se cargan en los marcos 4, 5, 6, 11 y 12. La Figura 7.10 muestra las diferentes tablas de páginas en este momento. Una tabla de páginas contiene una entrada por cada página del proceso, de forma que la tabla se indexe fácilmente por el número de página (iniciando en la página 0). Cada entrada de la tabla de páginas contiene el número del marco en la memoria principal, si existe, que contiene la página correspondiente, Adicionalmente, el sistema operativo mantiene una única lista de marcos libres de todos los marcos de la memoria que se encuentran actualmente no ocupados y disponibles para las páginas.

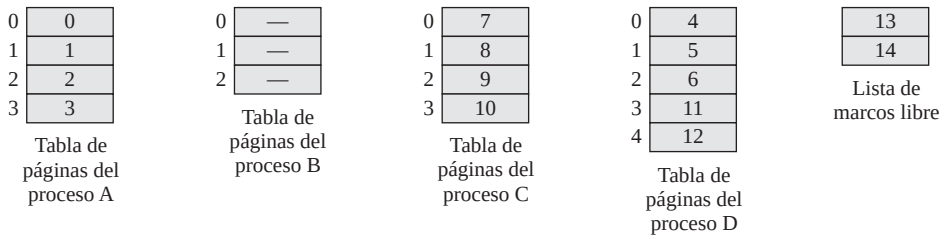


Figura 7.10. Estructuras de datos para el ejemplo de la Figura 7.9 en el instante (f).

Por tanto vemos que la paginación simple, tal y como se describe aquí, es similar al particionamiento fijo. Las diferencias son que, con la paginación, las particiones son bastante pequeñas; un programa podría ocupar más de una partición; y dichas particiones no necesitan ser contiguas.

Para hacer este esquema de paginación conveniente, el tamaño de página y por tanto el tamaño del marco debe ser una potencia de 2. Con el uso de un tamaño de página potencia de 2, es fácil demostrar que la dirección relativa, que se define con referencia al origen del programa, y la dirección lógica, expresada como un número de página y un desplazamiento, es lo mismo. Se muestra un ejemplo en la Figura 7.11. En este ejemplo, se utilizan direcciones de 16 bits, y el tamaño de la página es 1K = 1024 bytes. La dirección relativa 1502, en formato binario, es 0000010111011110. Con una página de tamaño 1K, se necesita un campo de desplazamiento de 10 bits, dejando 6 bits para el número de página. Por tanto, un programa puede estar compuesto por un máximo de 2^6=64 páginas de 1K byte cada una. Como muestra la Figura 7.11, la dirección relativa 1502 corresponde a un desplazamiento de 478 (0111011110) en la página 1 (000001), que forma el mismo número de 16 bits, 0000010111011110.

Las consecuencias de utilizar un tamaño de página que es una potencia de 2 son dobles. Primero, el esquema de direccionamiento lógico es transparente al programador, al ensamblador y al montador. Cada dirección lógica (número de página, desplazamiento) de un programa es idéntica a su dirección relativa. Segundo, es relativamente sencillo implementar una función que ejecute el hardware para llevar a cabo dinámicamente la traducción de direcciones en tiempo de ejecución. Considérese una dirección de $n+m$ bits, donde los n bits de la izquierda corresponden al número de página y los m bits de la derecha corresponden al desplazamiento. En el ejemplo (Figura 7.11b), n = 6 y m = 10. Se necesita llevar a cabo los siguientes pasos para la traducción de direcciones:

- Extraer el número de página como los n bits de la izquierda de la dirección lógica.
- Utilizar el número de página como un índice a tabla de páginas del proceso para encontrar el número de marco, k.
- La dirección física inicial del marco es $k \times 2^m$, y la dirección física del byte referenciado es dicho número más el desplazamiento. Esta dirección física no necesita calcularse; es fácilmente construida concatenando el número de marco al desplazamiento.

En el ejemplo, se parte de la dirección lógica 0000010111011110, que corresponde a la página número 1, desplazamiento 478. Supóngase que esta página reside en el marco de memoria principal 6 = número binario 000110. Por tanto, la dirección física corresponde al marco número 6, desplazamiento 478 = 0001100111011110 (Figura 7.12a).

Resumiendo, con la paginación simple, la memoria principal se divide en muchos marcos pequeños de igual tamaño. Cada proceso se divide en páginas de igual tamaño; los procesos más pequeños requieren menos páginas, los procesos mayores requieren más. Cuando un proceso se trae a la memo-

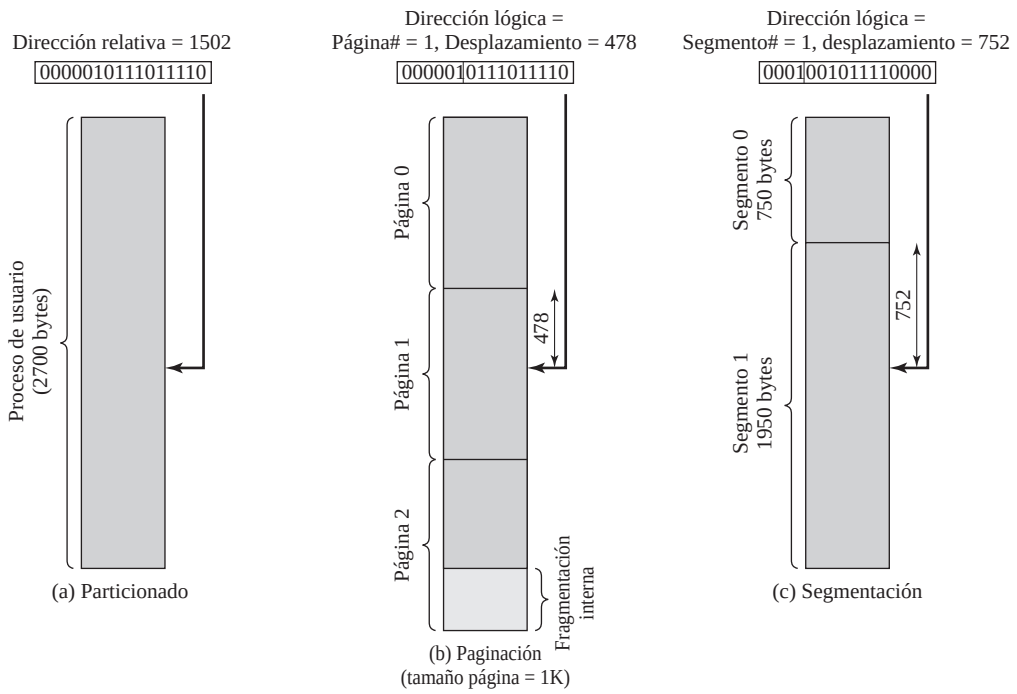


Figura 7.11. Direcciones lógicas.

ria, todas sus páginas se cargan en los marcos disponibles, y se establece una tabla de páginas. Esta técnica resuelve muchos de los problemas inherentes en el particionamiento.

7.4. SEGMENTACIÓN

Un programa de usuario se puede subdividir utilizando segmentación, en la cual el programa y sus datos asociados se dividen en un número de **segmentos**. No se requiere que todos los programas sean de la misma longitud, aunque hay una longitud máxima de segmento. Como en el caso de la paginación, una dirección lógica utilizando segmentación está compuesta por dos partes, en este caso un número de segmento y un desplazamiento.

Debido al uso de segmentos de distinto tamaño, la segmentación es similar al particionamiento dinámico. En la ausencia de un esquema de *overlays* o el uso de la memoria virtual, se necesitaría que todos los segmentos de un programa se cargaran en la memoria para su ejecución. La diferencia, comparada con el particionamiento dinámico, es que con la segmentación un programa podría ocupar más de una partición, y estas particiones no necesitan ser contiguas. La segmentación elimina la fragmentación interna pero, al igual que el particionamiento dinámico, sufre de fragmentación externa. Sin embargo, debido a que el proceso se divide en varias piezas más pequeñas, la fragmentación externa debería ser menor.

Mientras que la paginación es invisible al programador, la segmentación es normalmente visible y se proporciona como una utilidad para organizar programas y datos. Normalmente, el programador o compilador asignará programas y datos a diferentes segmentos. Para los propósitos de la programación modular, los programas o datos se pueden dividir posteriormente en múltiples segmentos. El in-

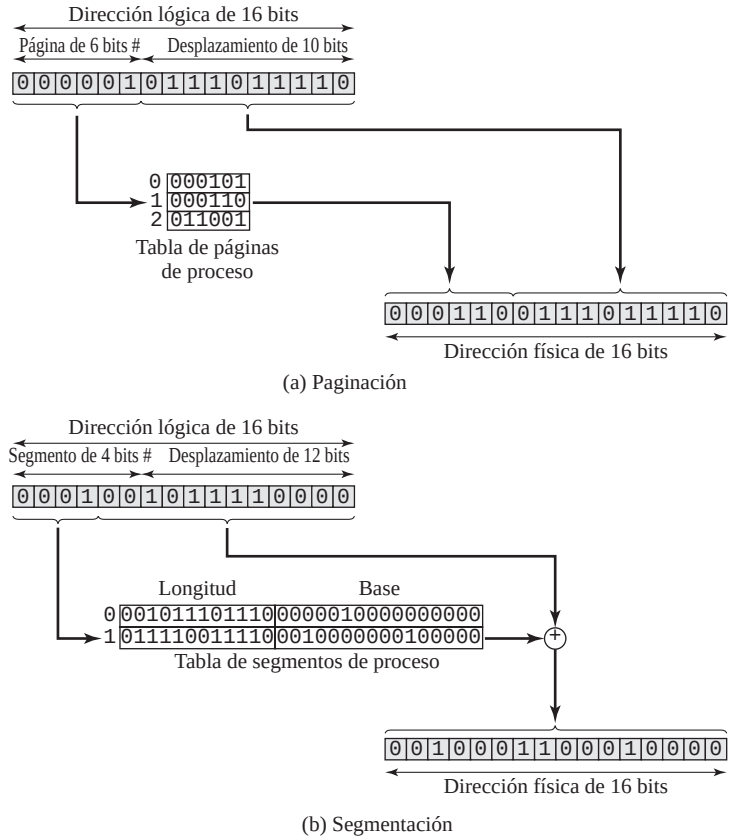


Figura 7.12. Ejemplos de traducción de direcciones lógicas a físicas.

conveniente principal de este servicio es que el programador debe ser consciente de la limitación de tamaño de segmento máximo.

Otra consecuencia de utilizar segmentos de distinto tamaño es que no hay una relación simple entre direcciones lógicas y direcciones físicas. De forma análoga a la paginación, un esquema de segmentación sencillo haría uso de una tabla de segmentos por cada proceso y una lista de bloques libre de memoria principal. Cada entrada de la tabla de segmentos tendría que proporcionar la dirección inicial de la memoria principal del correspondiente segmento. La entrada también debería proporcionar la longitud del segmento, para asegurar que no se utilizan direcciones no válidas. Cuando un proceso entra en el estado Ejecutando, la dirección de su tabla de segmentos se carga en un registro especial utilizado por el hardware de gestión de la memoria.

Considérese una dirección de $n+m$ bits, donde los n bits de la izquierda corresponden al número de segmento y los m bits de la derecha corresponden al desplazamiento. En el ejemplo (Figura 7.11c), $n = 4$ y $m = 12$. Por tanto, el tamaño de segmento máximo es $2^{12} = 4096$. Se necesita llevar a cabo los siguientes pasos para la traducción de direcciones:

- Extraer el número de segmento como los n bits de la izquierda de la dirección lógica.
- Utilizar el número de segmento como un índice a la tabla de segmentos del proceso para encontrar la dirección física inicial del segmento.

- Comparar el desplazamiento, expresado como los *m* bits de la derecha, y la longitud del segmento. Si el desplazamiento es mayor o igual que la longitud, la dirección no es válida.
- La dirección física deseada es la suma de la dirección física inicial y el desplazamiento.

En el ejemplo, se parte de la dirección lógica 0001001011110000, que corresponde al segmento número 1, desplazamiento 752. Supóngase que este segmento reside en memoria principal, comenzando en la dirección física inicial 0010000000100000. Por tanto, la dirección física es 0010000000100000 + 001011110000 = 0010001100010000 (Figura 7.12b).

Resumiendo, con la segmentación simple, un proceso se divide en un conjunto de segmentos que no tienen que ser del mismo tamaño. Cuando un proceso se trae a memoria, todos sus segmentos se cargan en regiones de memoria disponibles, y se crea la tabla de segmentos.

7.5. RESUMEN

Una de las tareas más importantes y complejas de un sistema operativo es la gestión de memoria. La gestión de memoria implica tratar la memoria principal como un recurso que debe asignarse y compartirse entre varios procesos activos. Para utilizar el procesador y las utilidades de E/S eficientemente, es deseable mantener tantos procesos en memoria principal como sea posible. Además, es deseable también liberar a los programadores de tener en cuenta las restricciones de tamaño en el desarrollo de los programas.

Las herramientas básicas de gestión de memoria son la paginación y la segmentación. Con la paginación, cada proceso se divide en un conjunto de páginas de tamaño fijo y de un tamaño relativamente pequeño. La segmentación permite el uso de piezas de tamaño variable. Es también posible combinar la segmentación y la paginación en un único esquema de gestión de memoria.

7.6. LECTURAS RECOMENDADAS

Los libros de sistema operativos recomendados en la Sección 2.9 proporcionan cobertura para la gestión de memoria.

Debido a que el sistema de particionamiento se ha suplantado por técnicas de memoria virtual, la mayoría de los libros sólo cubren superficialmente el tema. Uno de los tratamientos más completos e interesantes se encuentra en [MILE92]. Una discusión más profunda de las estrategias de particionamiento se encuentra en [KNUT97].

Los temas de enlace y carga se cubren en muchos libros de desarrollo de programas, arquitectura de computadores y sistemas operativos. Un tratamiento particularmente detallado es [BECK90]. [CLAR98] también contiene una buena discusión. Una discusión práctica en detalle de este tema, con numerosos ejemplos de sistemas operativos, es [LEVI99].

BECK90 Beck, L. *System Software*. Reading, MA: Addison-Wesley, 1990.

CLAR98 Clarke, D., and Merusi, D. *System Software Programming: The Way Things Work*. Upper Saddle River, NJ: Prentice Hall, 1998.

KNUT97 Knuth, D. *The Art of Computer Programming, Volume 1: Fundamental Algorithms*. Reading, MA: Addison-Wesley, 1997.

LEVI99 Levine, J. *Linkers and Loaders*. New York: Elsevier Science and Technology, 1999.

MILE92 Milenkovic,M. *Operating Systems: Concepts and Design*. New York: McGraw-Hill, 1992.